

Labo 01 — Réponses commentées

Chaque réponse suit le même schéma : la réponse, le mécanisme, la nuance ou le piège, un exemple vérifiable au terminal.

Question 1 — « Un conteneur, c'est une petite VM »

Réponse. Faux sur le point essentiel : un conteneur ne contient **pas de système d'exploitation** et n'a **pas son propre noyau**. C'est un processus de l'hôte, isolé par les *namespaces* et limité par les *cgroups*, exécuté par le noyau de l'hôte.

Pourquoi. Une VM démarre un noyau, puis un `init`, puis des dizaines de services système (journalisation, cron, SSH, réseau...) avant même que votre application ne démarre : d'où les secondes ou minutes de boot et les Go de disque. Un conteneur ne démarre rien de tout cela : le noyau tourne déjà, on lui demande juste de créer des namespaces et de lancer **un** processus. Le coût de démarrage est celui d'un `fork` + `exec`, quelques millisecondes ; le coût disque est celui des seules bibliothèques nécessaires à l'application.

→ LINUX

`fork` duplique le processus courant, `exec` remplace son contenu par un autre programme. C'est ainsi que naît *tout* processus Linux, `podman run` compris : Podman se duplique, le clone entre dans ses namespaces, puis exécute votre commande. Un conteneur naît exactement comme un `ls`.

Nuance. L'intuition « VM légère » n'est pas absurde pour un *utilisateur* : on obtient bien un `/`, un `hostname`, une IP, un `root`. Elle devient dangereuse dès qu'on parle sécurité : là où l'hyperviseur d'une VM est une vraie frontière, un conteneur partage le noyau — une faille noyau traverse cette frontière. Et sous Windows, la « légèreté » est relative : il y a bien une VM, WSL 2, mais une seule pour tous les conteneurs.

Exemple.

```
time podman run --rm alpine echo bonjour # ~0,3 s, dont l'essentiel est le pull/la CLI
podman run -d nginx:alpine               # ~64 Mo d'image, PID visible sur l'hôte via ps
```

Question 2 — 250 Mo « sans OS »

Réponse. L'image contient l'**espace utilisateur** (*userland*) d'une distribution : `/bin/sh`, `libc`, `coreutils`, les binaires PostgreSQL, la configuration. Ce qui est **absent**, c'est le **noyau** — et avec lui tout ce qui n'existe qu'au boot : chargeur de démarrage, `initrd`, modules noyau, `systemd`, pilotes, gestion du matériel.

Pourquoi. Un programme Linux appelle le noyau par des appels système (`open`, `read`, `fork`). Il n'a pas besoin d'embarquer un noyau, il lui suffit d'en trouver un : celui de l'hôte fait l'affaire. L'image ne fournit donc que ce qui manque au-dessus.

Nuance. C'est pourquoi une image « Alpine » et une image « Debian » peuvent tourner côte à côte sur le même Ubuntu WSL : ce sont trois userlands, un seul noyau. Et c'est aussi pourquoi une image Linux ne tourne pas sur un noyau Windows — d'où WSL.

Exemple.

```
podman run --rm alpine cat /etc/os-release | head -1 # NAME="Alpine Linux"
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # STRICTEMENT le même noyau
```

Question 3 — Deux conteneurs nginx, deux `ps` différents

Réponse. Le namespace `pid`. Chaque conteneur reçoit sa propre table de PID : son processus principal y est numéroté 1 et il ne peut voir aucun PID extérieur.

Pourquoi. Voir un processus est un prérequis pour agir dessus (`kill`, `/proc/<pid>`). En retirant la visibilité, le noyau retire de fait la capacité de nuire par cette voie. Sur l'hôte, ces processus existent bien, avec de vrais PID.

Nuance. Ce n'est pas de la sécurité « au sens strict » car ce n'est **pas une frontière infranchissable** : c'est une restriction de visibilité appliquée par le même noyau que celui du conteneur. Un conteneur lancé avec `--pid=host` ou `--privileged` retrouve la vue complète, et une faille noyau contourne le mécanisme. Le namespace isole ; il ne défend pas. En rootless, le namespace `user` ajoute une vraie barrière de *droits* par-dessus cette barrière de *vue*.

Exemple.

```
podman run -d --name web nginx:alpine
podman exec web ps -o pid,comm # PID 1 = nginx, puis ses workers
ps -ef | grep -c "[n]ginx" # sur l'hôte : les processus sont bien là
podman run --rm --pid=host alpine ps | head # l'isolation retirée : tout le WSL
podman rm -f -t 0 web
```

Question 4 — `Cannot connect to the Docker daemon`

Réponse. Chez votre collègue, le client a fonctionné : il a affiché sa version, puis a tenté d'ouvrir la socket `/var/run/docker.sock` pour interroger le **serveur**, et a échoué. Deux causes probables : (1) le daemon n'est pas démarré, (2) son utilisateur n'a pas la permission sur la socket. Chez vous, ce message est impossible : Podman n'a **pas de daemon** et ne contacte aucune socket ; chaque commande fait le travail elle-même, sous votre utilisateur.

Pourquoi. Toute commande Docker est un appel réseau vers `dockerd`. Modifier la commande ne changera rien, puisque personne ne l'a encore lue : le problème est en amont. Podman, lui, est un programme ordinaire : s'il se lance, il travaille.

Nuance. Il existe deux cas où Podman a un serveur : `podman --remote` (ou la variable `CONTAINER_HOST`) qui parle à un `podman system service` distant, et `podman machine` sous Windows/macOS, où le client Windows parle à une VM. Vous verriez alors `unable to connect to Podman socket`. Sous WSL avec Podman installé dans Ubuntu, vous n'êtes dans aucun de ces cas.

Exemple.

```
# Côté Docker, le diagnostic :
systemctl status docker # cause 1 : inactive (dead)
ls -l /var/run/docker.sock # srw-rw---- root docker : il faut être du groupe docker
# Côté Podman, la preuve qu'il n'y a rien à contacter :
podman version # un seul bloc "Client"
podman --remote version # Error: unable to connect to Podman socket ...
```

Question 5 — « Une image ne tourne pas »

Réponse. Une image est un ensemble de fichiers en lecture seule plus des métadonnées. Il n'y a aucun processus, aucun état, rien à ordonnancer : elle est aussi inerte qu'un `.zip` accompagné d'une notice.

Pourquoi. Au `podman run`, le moteur ajoute trois choses : (1) une **couche d'écriture** fine par-dessus les couches en lecture seule, (2) un jeu de **namespaces et de cgroups**, (3) l'**exécution** de la commande inscrite dans les métadonnées de l'image (`ENTRYPOINT` / `CMD`), via le runtime `crun`. Le résultat de cet assemblage est le conteneur.

Nuance. Le conteneur *créé* et le conteneur *démarré* sont deux étapes distinctes : `podman create` fait tout sauf lancer le processus, `podman start` le lance. `podman run` est simplement `create` + `start` (+ `pull` si l'image est absente).

Exemple.

```
podman create --name tmp alpine sleep 30 # conteneur créé, aucun processus
podman ps -a --filter name=tmp           # STATUS : Created
podman start tmp && podman ps             # STATUS : Up -> maintenant il tourne
podman rm -f -t 0 tmp
```

Question 6 — Données PostgreSQL après un `podman rm`

Réponse. Non, les données ont disparu. Et non, l'image n'a **pas** été modifiée : elle est strictement identique avant et après.

Pourquoi. Les écritures d'un conteneur (via *copy-on-write*) atterrissent dans sa couche d'écriture privée. `podman rm` supprime le conteneur **et** cette couche. Les couches de l'image sont en lecture seule : rien de ce que fait un conteneur ne peut les altérer — c'est ce qui garantit que deux conteneurs de la même image partent du même état.

Nuance. Deux corrections importantes. D'abord, `podman stop` ne détruit rien : un conteneur arrêté conserve sa couche, et `podman start` retrouve les données. C'est bien `rm` qui détruit. Ensuite, l'image officielle `postgres` déclare `/var/lib/postgresql/data` comme `VOLUME` : le moteur crée alors un volume **anonyme** qui survit au `rm` — mais sans nom, il est presque impossible à retrouver. En pratique, on considère les données perdues. Le volume nommé est l'objet du labo 06.

Exemple.

```
podman run -d --name c1 alpine sleep 600
podman exec c1 sh -c 'echo x > /marque.txt'
podman rm -f -t 0 c1
podman run --rm alpine ls /marque.txt # No such file or directory : l'image est intacte
```

Question 7 — Dix conteneurs, combien de disque ?

Réponse. Quelques dizaines de kilo-octets au total — pas 10×210 Mo. Chaque conteneur ne coûte que sa couche d'écriture, vide au départ, plus quelques fichiers de configuration (`hostname`, `resolv.conf` ...).

Pourquoi. Les couches de l'image sont **partagées** en lecture seule par tous les conteneurs qui en sont issus. Le pilote de stockage `overlay` superpose ces couches et une couche d'écriture vide par conteneur ; un fichier n'est copié vers cette couche qu'au moment où on le modifie (*copy-on-write*).

Nuance. La réponse change si chaque conteneur écrit beaucoup (logs, fichiers temporaires) : chaque modification d'un fichier de l'image en copie l'intégralité dans la couche du conteneur. Et un `podman ps -s` montre les deux chiffres : la taille propre et la taille « virtuelle ».

Exemple.

```
for i in 1 2 3; do podman run -d --name t$i alpine sleep 600; done
podman ps -s --format 'table {{.Names}}\t{{.Size}}' # 11.4kB (virtual 8.72MB) chacun
podman rm -f -t 0 t1 t2 t3
```

Question 8 — `root` dans le conteneur, `1000` sur l'hôte

Réponse. Le namespace `user` projette les identifiants du conteneur sur ceux de l'hôte : l'UID 0 du conteneur est votre UID 1000. Ce « root » ne possède, vis-à-vis du noyau et des fichiers de l'hôte, que vos droits : une tentative d'écrire dans `/etc/shadow` monté depuis l'hôte échoue avec `Permission denied`, exactement comme si vous le faisiez vous-même.

Pourquoi. Le noyau vérifie les permissions avec l'identité **réelle** (côté hôte), pas avec l'identité affichée dans le namespace. Les UID 1 à 65536 du conteneur sont projetés sur une plage réservée dans `/etc/subuid` (`100000-165535`) qui n'a aucun droit sur vos fichiers.

Nuance. Ce root est root à l'intérieur de ses namespaces : il peut installer des paquets, changer les permissions des fichiers de l'image, écouter sur le port 80 du conteneur. Ce qu'il ne peut pas, c'est franchir la frontière. Corollaire pratique : un fichier créé par le conteneur sous l'UID 999 (l'utilisateur `postgres`) apparaît sur votre hôte avec l'UID 100998 — c'est le piège classique des *bind mounts* en rootless (labo 06).

Exemple.

```
podman top veilleur user,huser # root / 1000
podman unshare cat /proc/self/uid_map # 0 -> 1000 (1), 1 -> 100000 (65536)
podman run --rm -v /etc:/hote alpine sh -c 'echo x >> /hote/shadow' # Permission denied
```

Question 9 — Le groupe `docker` et `sudo`

Réponse. Être membre du groupe `docker` donne le droit d'écrire sur `/var/run/docker.sock`, donc de faire exécuter n'importe quoi par un daemon qui tourne en **root** : `docker run -v /:/hote --privileged` donne l'hôte entier. C'est un `sudo` sans mot de passe, sans journalisation et sans limite. Avec Podman rootless, il n'y a ni daemon root ni socket : l'utilisateur ne peut rien faire de plus que ce qu'il pouvait déjà faire, et la règle n'a plus d'objet.

Pourquoi. L'audit exige de savoir *qui* a fait *quoi*. Une commande passée par la socket est exécutée par `dockerd`, sous l'identité `root`, sans trace liée à l'utilisateur. `sudo docker ...` laisse au moins une ligne dans `auth.log`. Podman rootless va plus loin : le conteneur est un processus de l'utilisateur, visible et attribuable dans `ps`.

Nuance. Podman rootless a un coût : pas de port < 1024 sans réglage, réseau en espace utilisateur un peu plus lent, certains montages et options interdits. En production, on rencontre

aussi Podman *rootful* (`sudo podman`), qui retrouve alors les mêmes précautions que Docker.

Exemple.

```
# Ce que permet le groupe docker (NE PAS FAIRE sur une machine partagée) :
docker run --rm -v /:/hote alpine cat /hote/etc/shadow      # lisible : le daemon est root
# La même chose sous Podman rootless :
podman run --rm -v /:/hote alpine cat /hote/etc/shadow     # Permission denied
```

Question 10 — Forme longue, forme courte

Réponse.

Raccourci	Forme longue	Objet
<code>podman ps -a</code>	<code>podman container ls -a</code>	conteneur
<code>podman images</code>	<code>podman image ls</code>	image
<code>podman rmi nginx:alpine</code>	<code>podman image rm nginx:alpine</code>	image
<code>podman rm web</code>	<code>podman container rm web</code>	conteneur

Pourquoi. `ps` et `images` datent des premières versions de Docker (2013), quand la CLI n'avait pas encore d'objets : `ps` imitait la commande Unix du même nom, `images` était un pluriel. La grammaire `objet action` est arrivée en 2017 (Docker 1.13), et Podman l'a reprise telle quelle. Les raccourcis sont conservés pour ne rien casser.

Nuance. La forme longue est la seule complète : `podman container ls`, `podman image ls`, `podman volume ls`, `podman network ls`, `podman pod ls` suivent le même patron, alors que `podman ps` n'a pas d'équivalent pour les volumes. En script, préférez la forme longue.

Exemple.

```
podman container ls -a --format '{{.Names}}'
podman image ls --format '{{.Repository}}:{{.Tag}}'
```

Question 11 — Deux `uname -r`, deux hôtes

Réponse. `uname -r` est un appel système : la valeur vient du **noyau**, jamais de l'image. Sous WSL, le noyau est `microsoft-standard-WSL2`, compilé par Microsoft pour la VM WSL ; sur un serveur Ubuntu natif, c'est le noyau `generic` du paquet Ubuntu. Un conteneur affiche le noyau de la machine qui l'exécute, quelle que soit l'image.

Pourquoi. Le conteneur est un processus du noyau hôte ; il n'y a pas de noyau dans l'image (question 2). Sous Windows, cet hôte n'est pas Windows mais la VM WSL 2.

Nuance. « Léger » reste vrai : la VM WSL est **unique**, démarrée une fois, et partagée par tous vos conteneurs ; ceux-ci restent des processus qui démarrent en millisecondes. Ce qui n'est plus vrai, c'est « pas de VM du tout ». Conséquences pratiques : la RAM disponible est celle de WSL (`.wslconfig`), et les fichiers Windows (`/mnt/c/...`) montés dans un conteneur sont lents, parce qu'ils traversent la frontière VM ↔ Windows. Travaillez dans le système de fichiers Linux (`~`).

Exemple.

```
uname -r # 6.6.87.2-microsoft-standard-WSL2
podman run --rm alpine uname -r # identique
podman info --format '{{.Host.Kernel}} {{.Host.MemTotal}}' # la RAM vue par WSL
```

Question 12 — Boucle infinie et RAM

Réponse. Non : le namespace `pid` ne fait que cacher les processus. Le mécanisme qui protège les voisins est le **cgroup** mémoire, activé par `--memory`. Sans limite, le conteneur consomme toute la RAM disponible ; quand le noyau n'a plus rien, l'**OOM killer** tue un processus de son choix — pas nécessairement le coupable.

Pourquoi. Namespaces et cgroups sont deux mécanismes indépendants : l'un isole la *vue*, l'autre plafonne la *consommation*. Avec `--memory=512m`, le dépassement provoque la mort du seul processus du conteneur (`Exited (137)`, `OOMKilled: true`), et le reste de la machine ne s'en aperçoit pas.

Nuance. En rootless, `--memory` n'est possible que si `systemd` délègue le contrôle `memory` à votre utilisateur — c'est le cas sur Ubuntu WSL une fois `systemd=true` activé. Et pour Java, une limite cgroup n'est utile que si la JVM la respecte : depuis Java 10, elle lit automatiquement le cgroup (`-XX:MaxRAMPercentage`), mais un `-Xmx` fixé trop haut à la main la dépassera quand même.

Exemple.

```
podman run -d --name limite --memory=128m --memory-swap=128m alpine sleep 600
podman stats --no-stream limite # MEM USAGE / LIMIT : ... / 134.2MB
podman inspect --format '{{.State.OOMKilled}}' limite # false - pour l'instant
podman rm -f -t 0 limite
```

Question 13 — Promouvoir une image sans la reconstruire

Réponse. Deux propriétés : l'**immuabilité** (une image identifiée par son digest ne change jamais) et le **standard OCI** (Docker et Podman produisent et lisent exactement le même format). Ce qui a été testé en recette est, bit pour bit, ce qui part en production, quel que soit le moteur. Reconstruire à chaque étape casse cette garantie : deux builds du même code ne donnent pas nécessairement la même image.

Pourquoi. Un build dépend de l'instant : `apt-get install` prend la version du jour, `FROM eclipse-temurin:21-jre` suit un tag mouvant, Maven résout des plages de versions. Entre le build de recette et celui de production, une dépendance peut avoir changé — et la validation en recette ne vaut plus rien.

Nuance. La promotion ne se fait pas en re-taguant `latest` mais en référençant le **digest** (`api@sha256:...`) ou un tag immuable (`api:1.4.2`). Le labo 02 y revient. Et le moteur importe peu : un `podman pull` d'une image poussée par `docker push` est un cas banal.

Exemple.

```
podman image inspect --format '{{.Digest}}' registry.interne/api:1.4.2
# Même digest sur le poste (Podman), en recette (Docker) et en production (Podman rootful).
```

Question 14 — `alias docker=podman`

Réponse. Vrai sans réserve pour : (1) tout le cycle image — `build`, `pull`, `push`, `tag`, `images`, `history`, `inspect`, les Dockerfiles ; (2) le cycle de vie des conteneurs — `run`, `ps`, `logs`, `exec`, `stop`, `rm` avec leurs options. Faux ou différent : (a) **pas de daemon** — pas de `docker.sock`, `--restart=always` ne survit pas à un reboot sans `systemd`, `podman rm -f` attend 10 s ; (b) **rootless** — pas de port < 1024 sans réglage, adresses IP absentes avec `pasta`, UID des fichiers de *bind mounts* décalés, `--memory` conditionné à la délégation cgroup.

Pourquoi. Podman a copié la *surface* de Docker (la CLI, le format) mais pas son *architecture*. Tout ce qui ne dépend que de la surface est identique ; tout ce qui touche à « qui exécute, avec quels droits, supervisé par qui » diverge.

Nuance. Les différences ne sont pas des défauts : chacune est la contrepartie d'un choix de sécurité. Et Docker Compose fonctionne avec Podman (`podman compose`, labo 09), au prix de quelques lignes de configuration.

Exemple.

```
alias docker=podman
docker run -d --name web -p 8080:80 nginx:alpine    # identique
docker run -d --name w80 -p 80:80 nginx:alpine      # Error: pasta failed ... Listen failed for
HOST TCP port */80: Permission denied
podman rm -f -t 0 web
```